TASK 1

Mar, 2026

PRE - GSOC TASK 1 DESIGN

Designing a Scalable GitHub Data Aggregation System

Presented By

Tarunya Kesharwani
github.com/tarunyaprogrammer

Completed Project

Git Repo - PreGSoC

1. Problem Overview

The Project-website displays information about projects hosted across 300+ GitHub repositories. These repositories constantly evolve — new commits, releases, contributors, and metadata updates occur frequently.

To keep the website accurate and useful, the system must:

- Automatically collect repository data from GitHub
- Detect updates when repositories change
- Serve repository information quickly to the frontend
- Minimize GitHub API usage due to strict rate limits
- Scale as the organization grows from hundreds to thousands of repositories

A naive approach (fetching GitHub data on every user request) would be inefficient and would quickly exceed GitHub API limits. Therefore, a dedicated aggregation system is required to collect, process, store, and serve repository data efficiently.

2. Design Goals

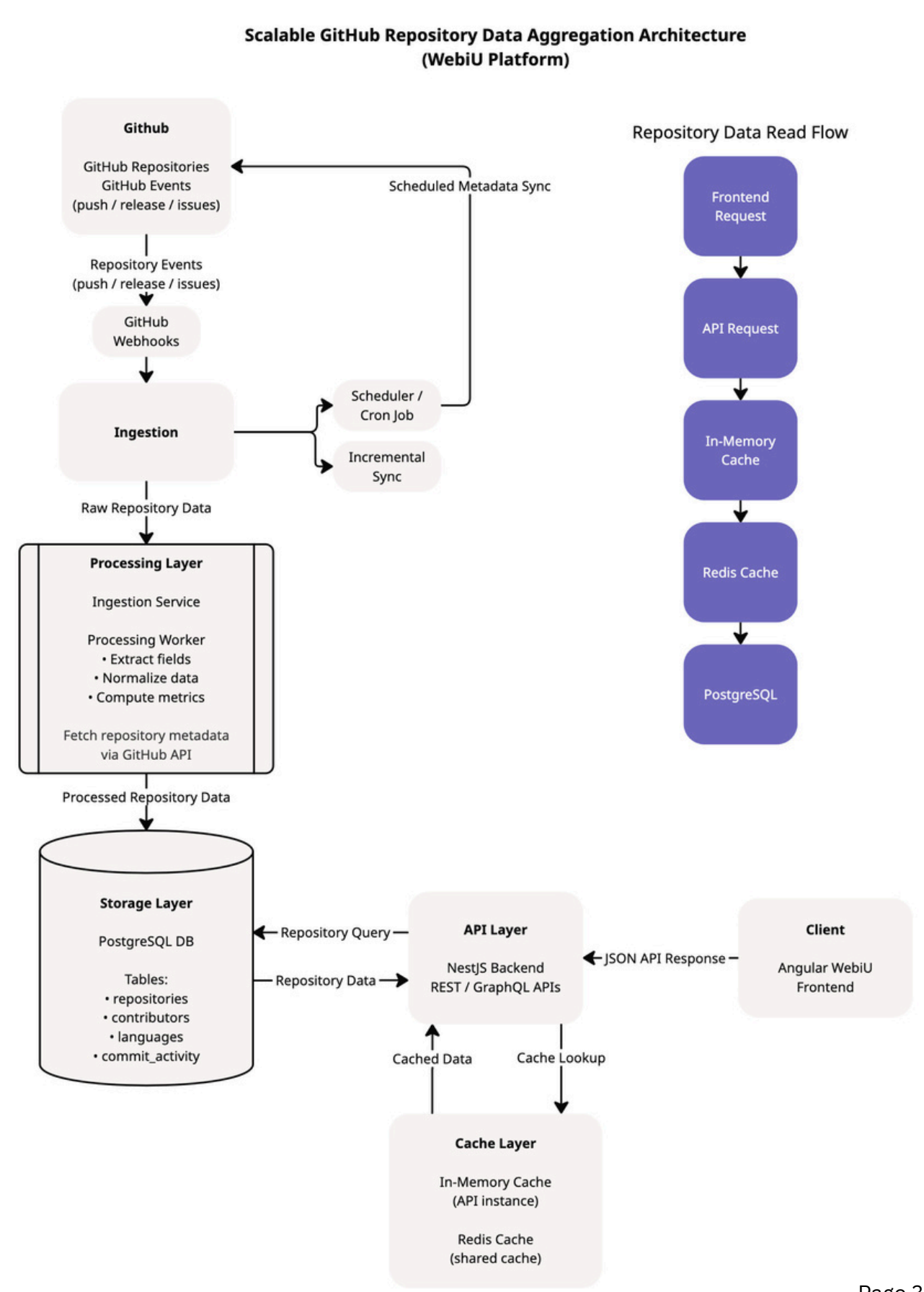
Before defining the architecture, it is important to clarify the goals guiding the design.

Goal	Reason
Minimize GitHub API usage	GitHub enforces strict rate limits (≈5000 requests/hour authenticated)
Automatic repository updates	Manual refresh should not be required
Fast frontend responses	Website pages should load quickly
Reliability	System must continue operating even if GitHub is temporarily unavailable
Scalability	Architecture should scale from **300 → 10,000 repositories**

These goals influence every architectural decision described below.

3. High Level System Architecture

The system follows a layered architecture where each layer performs a clear responsibility.



Why a Layered Architecture?

Separating responsibilities across layers provides several benefits:

- Maintainability — each layer focuses on a specific task
- Scalability — layers can be scaled independently
- Performance — caching and processing can be optimized separately

Layer	Responsibility
Data Ingestion	Collect repository updates from GitHub
Processing	Clean and structure the incoming data
Storage	Persist repository information
Cache	Improve response speed and reduce database/API load
API Layer	Serve data to the frontend

4. Data Ingestion Strategy

The system must detect repository changes as efficiently as possible.

Two complementary mechanisms are used.

Mechanism	Why It Is Used
GitHub Webhooks	Provide real-time updates when repository events occur
Scheduled Sync Jobs	Ensure no updates are missed if webhooks fail

Why Webhooks Are Primary?

GitHub webhooks provide event-driven updates. Instead of repeatedly polling the GitHub API, the system receives notifications whenever something changes.

This drastically reduces unnecessary API calls.

Webhook Security & Reliability

- Validate GitHub webhook signature
- Deduplicate events using event ID
- Retry failed processing via queue

Why Scheduled Sync Is Still Needed?
Webhook delivery is not guaranteed in rare cases (network errors or service downtime). A periodic synchronization job ensures that the system eventually reconciles any missed updates.

Example Webhook Events

- push
- release
- issue creation
- pull request updates

When such an event occurs, the system triggers a repository data refresh.

5. Processing Layer

Raw GitHub API responses contain many fields that are unnecessary for the WebiU website.
The processing layer transforms raw GitHub data into a clean and structured format before storing it.

Processing Task	Reason
Data extraction	Select only relevant fields
Data normalization	Ensure consistent schema
Filtering	Remove unnecessary fields
Metric calculation	Generate useful analytics values

Example Metrics

- number of contributors
- commit activity
- language usage

Why Processing Is Important?
Without a processing layer, the database would store large unstructured API responses, making queries inefficient and increasing storage usage.

Processing Worker Responsibilities?

- Normalize repository metadata
- Extract contributor statistics
- Compute activity metrics
- Detect technology stack
- Generate derived analytics

6. Storage Layer

Processed repository data is stored in a PostgreSQL database.

Why PostgreSQL?

Feature	Benefit
Relational schema	Efficient for structured repository data
Strong indexing	Faster queries at scale
JSONB support	Flexible storage of GitHub metadata

This combination allows both structured queries and flexible data storage.

Data Storage Strategy

The system separates frequently used structured repository data from large or rarely requested data to optimize storage and API usage.

Data Category	Examples	Reason
Persisted Repository Data	Repository metadata, stars, forks, open issues, contributors, languages, commit activity	Required frequently for displaying repository information on the website
Dynamically Fetched Data	Full repository file tree, rarely requested analytics	Large or rarely accessed data that would unnecessarily increase storage and synchronization overhead

These fields are fetched dynamically to avoid unnecessary storage overhead.

7. Rate Limit Strategy

GitHub imposes strict API rate limits, making efficient request usage critical.

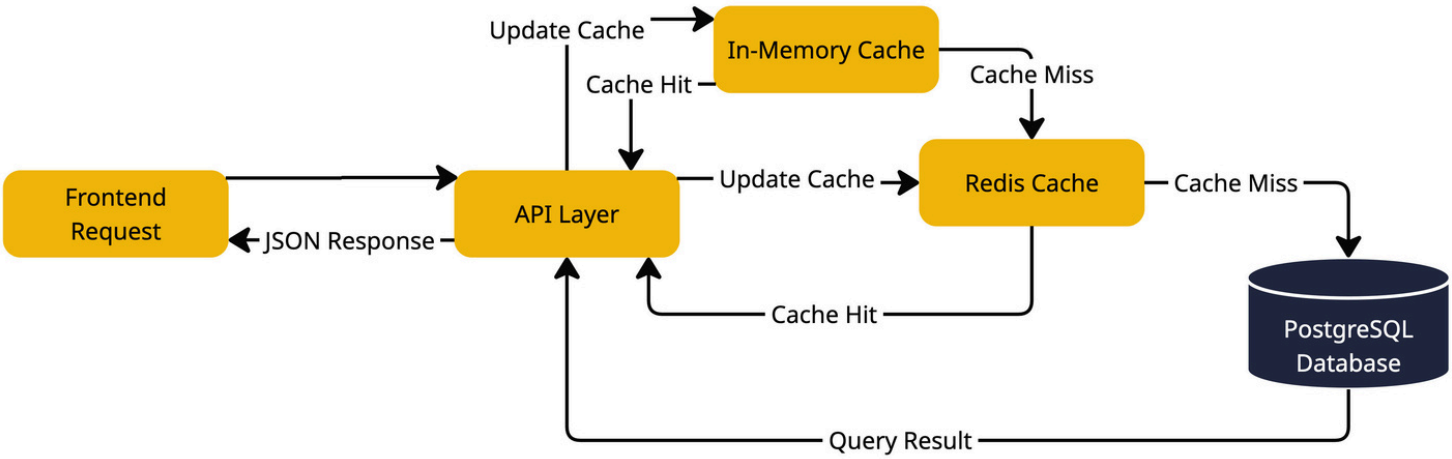
Strategy	Why It Helps
Github GraphQL Batch Queries	Fetch multiple repository fields in one request
Conditional Requests	Avoid unnecessary responses using ETag
Incremental Updates	Refresh only repositories that changed
Backoff Strategy	Prevent excessive requests when limits approach

GraphQL Advantage

Github-GraphQL allows multiple fields and repositories to be fetched in a single request, significantly reducing the total number of API calls.

8. Caching Strategy

Caching plays a critical role in improving performance and reducing load.



Why Multiple Cache Layers?

Different cache layers serve different performance needs.

Cache Layer	Purpose	TTL
In-Memory Cache	Fastest access within a single API instance	1–5 minutes
Redis Cache	Shared cache across multiple servers	30–60 minutes
CDN Cache	Faster responses for public requests	several hours

Cache invalidation triggered by webhook updates.

Benefits

- faster API responses
- reduced database queries
- reduced GitHub API usage

9. Scalability Plan

The system is designed to scale as repository count increases.

Repository Count	Infrastructure Strategy
300	Single API instance + Redis
1,000	Add background processing workers
5,000	Multiple API servers + load balancer
10,000	Distributed workers + DB read replicas

Key Scaling Techniques

- horizontal API scaling
- database indexing
- background workers

These allow the architecture to handle both higher repository counts and higher traffic.

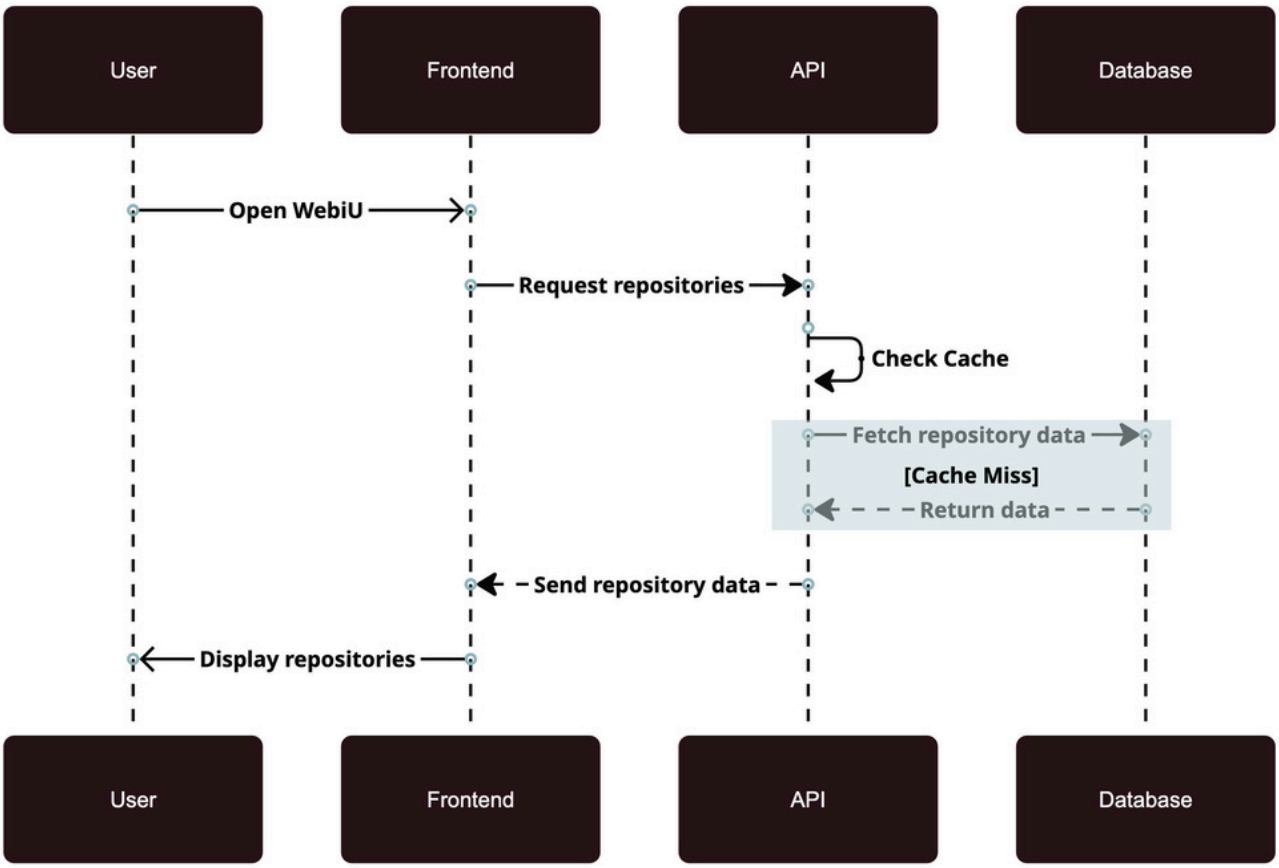
10. Failure Handling

External services such as GitHub may occasionally fail or become unavailable.

Failure Scenario	System Response
GitHub API outage	Serve cached data
Rate limit exhaustion	Pause ingestion and retry later
Repository deleted	Mark repository inactive
Database outage	Temporary Redis-only responses
Worker crash	Reprocess job from queue

These mechanisms ensure the website continues functioning even during failures.

11. API Flow



The caching layers ensure most requests are served quickly without querying the database.

12. Technology Choices

The architecture aligns with the existing WebiU stack.

Technology	Purpose	Reason
Angular	Frontend framework	Already used by WebiU
NestJS	Backend API framework	Modular and scalable
PostgreSQL	Database	Strong querying and indexing
Redis	Caching layer	Extremely fast read performance
GitHub APIs	Data source	Official GitHub data access

Using technologies already present in WebiU reduces integration complexity and improves maintainability.

Conclusion

This architecture provides an efficient system for aggregating GitHub repository data while minimizing API usage and ensuring scalability. By combining event-driven updates, structured processing, layered caching, and scalable infrastructure, the system can reliably support the WebiU website as repository counts grow from hundreds to thousands.

The design ensures:

- automatic repository updates
- fast frontend responses
- efficient GitHub API usage
- scalability for future growth